

Document number: D1791r0
Date: 2019-06-16
Project: Programming Language C++
Audience: SG1
Reply-to: Christopher Kohlhoff <chris {at} kohlhoff_dot_com>
Jamie Allsop <jamie.allsop {at} clearpool_dot_io>

Evolution of the P0443 Unified Executors Proposal to accommodate new requirements

Introduction

This paper proposes an updated consensus design for executors as a revision of the existing consensus paper [A Unified Executors Proposal for C++, P0443r10](#) which incorporates, or explicitly addresses as appropriate, new requirements which have emerged since that paper was produced. The follow-up to this paper would be a revision 11 of [P0443r10](#).

Recap on what P0443 presents

[P0443](#) establishes:

- core concepts to represent task submission, and
- facilities to allow interoperability between otherwise disparate domain models that require executors.

Importantly, [P0443](#) does not seek to specify high level control structures (such as algorithms), but rather provide a foundation for building these structures.

This is achieved by standardising a vocabulary of well-known [properties](#) related to execution. This in itself is extensible to accommodate future expansion of this standardised vocabulary, and also facilitates the expression of domain-specific, and indeed application-specific, requirements.

Framing the discussion

The debates and efforts surrounding the standardisation of executors has been a long journey with many participants, starting in earnest back in 2012, but has origins well before that with different use cases motivating disparate views on what the “one true” executor abstraction should look like.

A number of distinct proposals emerged from that initial work, each with important but differing views on the best way forward, motivated by their different use cases and core requirements. At that time one option could have been to allow each to continue evolution in relative isolation with a view to eventual standardisation of multiple related, but distinct, executor abstractions. However, the view of SG1 was that a single executor abstraction was preferred, and so direction was given to the authors of the various proposals to make an effort to find a consensus proposal that would allow the standardisation of that single, or more precisely, unified abstraction.

After many months of discussion and debate an initial revision of [A Unified Executors Proposal for C++, P0443](#) was produced, in late 2016. Then in 2017 the [Executors Design Document](#) was produced as a companion document to [P0443](#) to capture and document the design decisions that underpinned the work in [P0443](#).

Since that [initial version](#), [P0443](#) has undergone 10 revisions, taking onboard the views from a large body of experts representing both their respective view points, their industries and domains, and their companies. This has included such organisations and groups as Google, Sandia National Labs, Codeplay, Facebook, Nasdaq, Clearpool.io, Nvidia, Stellar Group, Microsoft, RedHat, HPX, HPC, and domains such as Ultra Low Latency Finance, Embedded Systems, APU, GPU, SYCL, Networking, Machine Learning, Library Development and Big Data to name a few.

In addition many more have influenced the paper through discussions at standards meetings and other mediums. In all more than 100 papers and revisions have been produced that either directly or indirectly have significantly impacted the consensus position represented by [P0443](#).

In summary [P0443](#) represents a significant body of compromise and consensus seeking. When the parties represented by [P0443](#) began their journey, we all saw our own requirements as essential and therefore universally applicable. As a result initial compromises were focused on gaining buy-in from other parties that those requirements were in fact universal and should be accepted as such. A significant part of the consensus building experience was to realise that while they may be essential in a particular domain, they were not universal and so some method of satisfying that reality needed to be found.

As a result [P0443](#) represents a unified set of building blocks and the tools to assemble them, and not, as some may prefer, a set of high level control structures. The intention is that those high level control structures, which are likely to be more domain specific, should be layered on top.

In more recent times (mid 2018) we have had additional requirements put forward starting with paper [P1055](#), with the culmination of those additional requirements to be found in [P1660](#) - found in the same mailing as this paper.

The authors of this paper believe that these requirements represent additional evidence of the value of the approach stipulated in [P0443](#) and this paper will demonstrate how those requirements can be met using the facilities of [P0443](#).

This is important because there is a danger that the specific needs of [P1660](#) will inadvertently be given precedence over the existing and extensive compromises that have been made in order to reach the consensus position that is [P0443](#). In other words we should, if at all possible, identify if the requirements identified in [P1660](#) can be satisfied by [P0443](#) and then if we cannot satisfy them directly we should explore what is deficient with [P0443](#) that makes that the case and then address those deficiencies as requirements for [P0443](#).

Said another way the default starting point should be to assume that the status quo consensus can meet the requirements of [P1660](#), and then investigate how, and whether they can (or cannot) do so.

In fact the properties mechanism that was surfaced as part of the consensus journey of [P0443](#) was developed precisely to address this situation: that not all domains have the same requirements but they need a way to express those requirements as first class vocabulary elements in their executor domain models.

The approach that most of the parties take is to express these additional requirements as properties, and to encapsulate them in a single place using a domain-specific control structure. Thus the approach for integrating new use cases into the [P0443](#) executor framework should be to:

- Identify what additional properties, if any, are required to support the domain specific requirements
- Define domain-specific control structures that encapsulate the use of those properties

Feedback from Rapperswil was that [P0443](#) had too many concepts, and that we should find a way to minimise those concepts. This paper represents the end of that journey with simplification to a single Executor concept (which incidentally is inline with one of the requirements identified in [P1660](#)). Specifically the `OnewayExecutor` concept renamed to simply `Executor`, is now the consensus based fundamental executor concept.

The requirements gap with P1660

In essence [P1660](#) elaborates the requirements for executors as a low-level primitive for a Sender/Receiver framework and therefore it is these requirements that we need to explore the viability of in the context of [P0443](#).

Through assessing [P1660](#) in this context, and supplemented by clarifications with the authors, we can observe that the following requirement gaps are now closed:

- oneway executor as a primitive for Sender/Receivers. [P1660](#) recognises that this is a suitable primitive.

However the following requirements exist and are not aligned directly with [P0443](#). Namely that [P1660](#) requires:

- A channel for signalling when an error occurs after task submission but prior to task execution.
- A channel to signal when a task is unstaged without execution.

The motivation behind these two requirements is to make it easier to support the higher level Sender/Receiver design in a way that better satisfies their executor usage requirements.

The proposal presented in [P1660](#) then seeks to address these requirements by:

- Defining concepts `CallbackSignal` and `Callback` that associate an error channel with a submitted task.
- Specifying that all models of the Executor concept must be aware of these concepts. If a submitted function object models `Callback` then the Executor shall detect this at compile time and deliver the two signals described above to the function object.

This approach imposes a burden on all Executor types in order to satisfy these new requirements. This runs counter to the experience of developing consensus in the design of [P0443](#), where a key lesson was that while we each have requirements that are key to ones own use cases, these requirements are not universal.

While we recognise the validity of these new requirements, we believe that they can and should be addressed using the existing facilities and approaches contained with [P0443](#).

P0443 can support the additional requirements of P1660

To qualify earlier statements we believe that we will want to define a control structure as the primary interface for user experience (where user includes authors of algorithms). Let us say that this new control structure is a function named `submit`. This is in line with [P1660](#). For exposition only (readers should refer to [P1660](#) for a more detailed discussion):

```
template<Callback C, Executor<C> E>
void std::tbd::submit(E&& e, C&& c);
```

The question then is how would this be provided in the context of [P0443](#)? In fact there are several approaches that could be adopted (or combination of approaches) that would offer this higher level control structure while satisfying:

- the needs outlined in [P1660](#), namely the support for error and done signals, and,
- the general design goals of [P0443](#), namely that domain specific needs should not be forced onto other domains unnecessarily.

More specifically there are at least four approaches to provide the necessary support for this control structure. We could:

1. Define a property to represent each interesting callback signal.
2. Define an “enumerated” property set to describe possible error handling strategies.

3. Define `submit` as a customisation point object, with an associated property type to allow the customisation to be propagated through a polymorphic executor wrapper.
4. Define a callback-based executor as its own concept, and use `require_concept` to provide the means to generically convert executors to this concept.

All of these are viable implementation approaches, however the first two are most consistent with the design of [P0443](#) that encourages employing properties as a mechanism to build a vocabulary or toolkit.

Approach 1: define a property to represent each signal

In this implementation approach, each interesting signal is represented by a corresponding property. For example, an error notification signal would have a property `on_error_t` that is specified as follows:

```
template <class Handler>
struct on_error_t
{
    template <class T>
        static constexpr bool is_applicable_property_v;

    static constexpr bool is_requirable = true;

    template <class E>
        static constexpr auto static_query_v
            = Executor::query(allocator_t);

    constexpr on_error_t();
    template <class OtherHandler>
        constexpr on_error_t(const on_error_t<OtherHandler>& other);
    constexpr explicit on_error_t(Handler h);

    template <class OtherHandler>
        constexpr on_error_t<OtherHandler> operator()(OtherHandler h) const

    constexpr Handler value() const;

private:
    Handler handler_; // exposition only
};

constexpr on_error_t</* ... */> on_error;
```

This property is used to associate a handler for the error signal. To satisfy the requirements embodied by [P1660](#), this handler must be noexcept invocable, and when invoked is passed:

- a reference to the submitted function object; and
- the error

This gives the user an opportunity to propagate the error directly to the submitted task, or perhaps to instead treat error handling as a cross cutting concern.

```
auto ex1 = pool.executor();

auto ex2 = std::require(ex1,
    execution::on_error(
        [](auto& f, auto e) noexcept
        {
            f.error(e);
        }
    )
);
```

```

struct my_task
{
    void operator()();
    template <class E> void error(E) noexcept;
    void done() noexcept;
};

ex2.execute(my_task{});

```

A similar property `on_done_t` could be specified for the “done” signal.

To support polymorphic wrappers, these properties have counterparts `polymorphic_on_error_t` and `polymorphic_on_done_t` that perform the necessary type erasure to propagate the signals through a polymorphic wrapper. These properties would automatically convert to their type-safe equivalents, so that users only need to use the type-safe property, as shown below:

```

execution::executor<
    execution::polymorphic_on_error_t,
    /* other properties */
> ex3 = ex2;

auto ex4 = std::require(ex1,
    execution::on_error(
        [](auto& f, auto e) noexcept
        {
            f.error(e);
        }
    )
);

ex4.execute(my_task{});

```

This approach is similar to that already used in [P0443](#) for the allocator property. Other examples of signals that might use this approach include: polling for task cancellation, flow control (such as overflow and underflow events), and exception reduction functions.

Approach 2: define a set of properties

An alternative approach is to provide a set of enumerated properties that represent a set of available error handling strategies for Executors. This may include “enumerator” properties with names such as `unspecified` or `terminate`.

One of those property “enumerators” may be called `propagate_to_callback` which, if present, indicates that the Executor will test for some well-known concept `callback` and, if the concept is detected, capture errors and propagate them to the callback.

[P0797, Handling Concurrent Exceptions with Executors](#) explores this design space further.

Note that both the approaches outlined above are not necessarily mutually exclusive.

Using property-based submit in generic code

As discussed in [P0761, the Executors Design Document](#), and reiterated above we expect that most users will not interact with executors directly, but rather through some kind of control structure (of which algorithms may be an instance).

With that in mind, users who wish to traffic in `callback` types (as described in [P1660](#)), would submit their callbacks for execution using either their own, or as [P1660](#) implies, a standardised `submit` function. A simplistic

exposition of submit would be:

```
template <Executor E, Callback C>
void submit(E e, C c)
{
    std::require(std::move(e),
        std::execution::on_error(
            [](auto& c, auto e)
            {
                c.error(e);
            }
        ),
        std::execution::on_done(
            [](auto& c)
            {
                c.done();
            }
        )
    ).execute(std::move(c));
}
```

As `on_error` and `on_done` are specified as properties, whether they are supported for a particular Executor is an explicit opt-in on the part of that Executor.

Furthermore, as a user of executors we are able to test for their supportability on a particular Executor by using `can_require_v`. A more complete exposition of submit might therefore be:

```
template <Executor E, Callback C>
void submit(E e, C c)
{
    auto error_handler = [](auto& c, auto e) noexcept
    {
        std::move(c).error(std::move(e));
    };

    auto done_handler = [](auto& c) noexcept
    {
        std::move(c).error();
    };

    if constexpr (
        std::can_require_v<E, std::execution::on_error_t<decltype(error_handler)>> &&
        std::can_require_v<E, std::execution::on_done_t<decltype(done_handler)>>
    )
    {
        std::require(std::move(e),
            std::execution::on_error(error_handler),
            std::execution::on_done(done_handler)
        ).execute(std::move(c));
    }
    else
    {
        // Fallback to an alternative method...
    }
}
```

This second formulation would allow `submit` to be used generically across all executor types, but from a pure user experience viewpoint they traffic in the vocabulary of `submit`, while executor authors are free to establish whatever constraints or freedoms they require to satisfy their domain requirements and provide a custom `submit` (or other more appropriate control structure) should the standard `submit` not meet their needs.

Standardising `submit`, `CallbackSignal` and `Callback`

While the specification of the submit control structure is outside the scope of this paper, we expect that it, and its supporting concepts `CallbackSignal` and `Callback`, would be standardised as vocabulary elements to support the development of generic algorithms that rely on this behaviour, as inferred by [P1660](#).

Outline of proposed changes to P0443 for a Revision 11

These changes are relative to [P0443r10](#):

- Remove the existing `is_executor` trait, the `is_executor_v` variable template, and all `is_executor_v` members.
- Rename the `oneway_executor` type requirements to `Executor`.
- Rename the `is_oneway_executor` trait to `is_executor`, and corresponding `is_oneway_executor_v` variable template to `is_executor_v`.
- Remove the `is_bulk_oneway_executor` trait and `is_bulk_oneway_executor_v` variable template.
- Remove the interface changing properties `oneway_t` and `bulk_oneway_t`.
- Consolidate the polymorphic wrapper in its oneway form as class template `executor<>`.
- Remove the `require_concept` member, and all uses of `can_require_concept_v`, from the polymorphic wrapper.
- Remove the `static_executor_cast` member from the polymorphic wrapper.
- Combine the specification for `static_thread_pool` executor types with the `execution::oneway` property into the general specification of all `static_thread_pool` executor types.
- Remove the specification for `static_thread_pool` executor types with the `execution::bulk_oneway` property.
- Add a new property `on_error_t`.
- Add a new property `polymorphic_on_error_t`.
- Add a new property `on_done_t`.
- Add a new property `polymorphic_on_done_t`.

Conclusion

In summary this paper seeks to:

- reaffirm the importance of recognising the extensive consensus and compromise embodied in the [Unified Executors Proposal for C++, P0443r10](#) paper.
- recap the value and benefits of seeking to accommodate new requirements against the existing extensibility facilities offered by [P0443](#) whose primary purpose is to facilitate this kind of adaption and customisation.
- assert that while the additional requirements of [P1660](#) have recognisable value for the target use cases they do, as implemented in [P1660](#), impose unnecessary burden on executor authors of other domains. More directly, as implemented they break one of the key design goals of [P0443](#) which was specifically to support scenarios like this where it is recognised not all requirements are, or should be treated as, universal.
- demonstrate that the requirements of [P1660](#) can be met by [P0443](#) with minor extensions to the vocabulary of standardised properties and without breaking the design goals of [P0443](#).

The next steps would be to issue a new revision of P0443 that reflects these changes, and to work with the authors of P1660 to specify standardese for submit and related control structures.